

Product Requirements Document (PRD): Lightweight Real-Time Chat App

Product Name: EchoChat (*Working Title*) **Document Version:** 1.0 **Target Release:** MVP (Phase 1) **Document Status:** Ready for Review

1. Executive Summary & Problem Statement

1.1 Summary

EchoChat is a privacy-first, lightweight messaging platform that enables instant real-time communication between users with zero personally identifiable information (PII) required. Authentication relies strictly on a unique username and password—completely eliminating mandatory email addresses, phone numbers, or third-party OAuth providers.

1.2 Problem Statement

Modern messaging platforms (WhatsApp, Telegram, Signal) tie user identity directly to mobile phone numbers or email addresses. This introduces multiple user pain points:

- **Privacy concerns:** Reluctance to reveal personal phone numbers or emails to casual contacts, gaming groups, or ad-hoc collaborators.
- **Friction in onboarding:** SMS verification codes, country code requirements, and carrier delays create unnecessary signup drop-offs.
- **Identity fragmentation:** Users often cannot maintain separate, disposable, or context-specific identities across disparate social or professional domains.

1.3 Vision & Value Proposition

Provide an instant, frictionless, and anonymous messaging experience: sign up in seconds, share a handle, and communicate in real time with end-to-end transport security.

2. Goals & Non-Goals

2.1 Goals (MVP)

- **Zero-PII Onboarding:** Create an account and log in using only an alphanumeric username and password.
- **Sub-Second Real-Time Delivery:** Deliver text messages with < 100ms transport latency under standard network conditions via WebSockets.
- **Direct 1-on-1 Messaging:** Search and initiate direct messaging conversations by exact username.
- **Delivery Guarantees & Receipts:** Track message status (*Sent, Delivered, Read*) and presence status (*Online, Offline, Typing...*).
- **Session Durability:** Persistent authentication sessions across browser refreshes or app

reopens without re-prompting credentials.

2.2 Non-Goals (Out of Scope for MVP)

- SMS/Email-based password reset or OTP verification.
- Address book contact sync (native phone contacts).
- Voice and video calling (WebRTC media streams deferred to Phase 3).
- Large media transfers / video transcoding (MVP supports text and standard emojis only; basic image sharing deferred to Phase 2).

3. User Personas & Core Use Cases

Persona	Motivation	Primary Need
Privacy Advocate (Alex)	Wants private conversations without linking cellular or email identity.	Zero PII collection, strictly anonymous credentials, client-side safety.
Gamer / Temporary Collaborator (Sam)	Wants a quick channel to coordinate a raid or project without sharing personal contacts.	Instant signup (< 30 seconds), quick username exchange, low latency.
Disposable / Secondary Identity User (Morgan)	Needs a segregated communication channel separate from personal/work accounts.	Rapid switching, easy account abandonment without credential baggage.

4. User Authentication & Account Management

Because there is no email or phone number to verify, account creation must protect against abuse while managing the impossibility of conventional password recovery.

4.1 Registration & Signup Flow

- **Input Fields:**
 - Username (Required)
 - Password (Required)
 - Confirm Password (Required)
 - Optional: CAPTCHA challenge (Cloudflare Turnstile or invisible hCaptcha) to prevent automated bot registration.
- **Validation Rules:**
 - **Username:** 3–24 characters, alphanumeric with underscores and hyphens (`^[a-zA-Z0-9_-]{3,24}$`). Case-insensitive uniqueness enforced at database level.
 - **Password:** Minimum 8 characters. Must contain at least one uppercase letter, one lowercase letter, and one digit. Passwords hashed using Argon2id or bcrypt (cost factor `\ge 12`).
- **Zero PII Confirmation:** UI must clearly state: *"No email or phone number is stored. We cannot recover your account if you forget your password."*

4.2 Account Recovery Strategy (No-Email Alternatives)

Traditional "Forgot Password" links will not work. The following recovery mechanisms are specified:

1. **Cryptographic Recovery Key (Fallback Code):**
 - Upon successful signup, the server generates a one-time 24-character cryptographic recovery code or 12-word mnemonic seed.
 - The user must confirm/download this key before completing registration.
 - If the password is forgotten, entering the Username + Recovery Key allows resetting the password.
2. **Account Deletion / Purge:**
 - If both password and recovery key are lost, the user can choose to wait for an inactive purge (e.g., 90 days of inactivity) or re-register a new handle.

4.3 Login & Session Management

- **Authentication:** REST API endpoint POST /api/v1/auth/login returning an HTTP-only, secure, SameSite=Strict cookie containing a signed JWT or session token.
- **Token Lifecycle:**
 - Access Token: Short-lived (15 minutes).
 - Refresh Token: Long-lived (30 days), stored securely with token rotation.
- **Concurrent Sessions:** Support multiple active sessions per user (e.g., desktop browser and mobile web). Ability to "Log Out of All Other Sessions."

5. Functional Requirements: Real-Time Messaging

5.1 Real-Time Connection

- **Protocol:** WebSocket (WSS) authenticated via initial handshake containing the user's valid session token.
- **Heartbeat / Keep-Alive:** Periodic ping/pong packets every 30 seconds to maintain NAT traversal and detect connection dropouts.
- **Auto-Reconnect:** Exponential backoff reconnection strategy on network loss (1s, 2s, 4s, 8s up to 30s max).

5.2 Direct Messaging (1-on-1)

- **User Discovery:**
 - Search bar accepting exact or prefix username matching.
 - Privacy setting: Toggle whether a user is discoverable via partial search or exact match only.
- **Conversation Threading:**
 - One unique conversation record per user pair.
 - Chronological message stream sorted by timestamp.
- **Message Payloads:**
 - Maximum text length: 4,000 characters per message.

- UTF-8 encoding supporting full Unicode emoji spectrum.

5.3 Message Lifecycle & Status Receipts

Each message transitions through four deterministic states represented in the UI:

1. **Sending:** Message queued locally, awaiting server ACK over WebSocket.
2. **Sent (Single Checkmark):** Message persisted to the database and acknowledged by server.
3. **Delivered (Double Checkmarks):** Recipient's active client received the message payload.
4. **Read (Colored Double Checkmarks):** Recipient opened the conversation view containing the message.

5.4 Presence & Typing Indicators

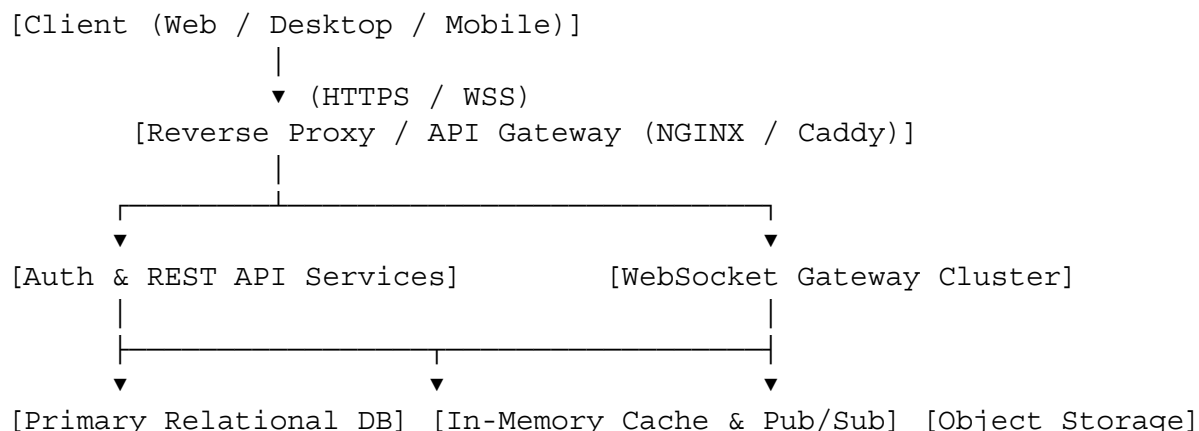
- **Online / Offline Status:**
 - Tracked via connection state in an in-memory cache (Redis).
 - Transition to Offline immediately upon clean disconnect, or after 45 seconds of missed heartbeats.
- **Typing Indicator:**
 - Ephemeral event sent over WebSocket when a user enters keystrokes in the composer.
 - Automatically expires 3 seconds after the last keystroke or upon sending the message.

5.5 Message History & Pagination

- **Infinite Scrolling:** Cursor-based pagination fetching 50 messages per batch (GET /api/v1/conversations/{id}/messages?before_id={cursor}).
- **Local Caching:** Client-side caching (IndexedDB in web browsers) to instantly render prior conversations before fetching deltas from the network.

6. Technical Architecture & Data Model

6.1 High-Level Architecture



(PostgreSQL)
Phase 2)

(Redis / Dragonfly)

(S3 / MinIO -

- **WebSocket Gateway:** Handles persistent socket connections and routes incoming events.
- **Redis Pub/Sub:** Coordinates message delivery across multi-node WebSocket servers when participants are connected to different socket instances.
- **PostgreSQL:** System of record for users, conversations, and message history.

6.2 Database Schema (Core Tables)

users

Column	Type	Constraints	Description
id	UUID	Primary Key, default gen_random_uuid()	Unique user ID
username	VARCHAR(24)	Unique, Not Null, Indexed (Lower)	Case-insensitive handle
password_hash	TEXT	Not Null	Argon2id hash string
recovery_hash	TEXT	Not Null	Hashed recovery key
display_name	VARCHAR(32)	Nullable	Optional alias shown in chat
created_at	TIMESTAMPTZ	Default NOW()	Account creation time
last_seen_at	TIMESTAMPTZ	Nullable	Presence tracker

conversations

Column	Type	Constraints	Description
id	UUID	Primary Key	Unique conversation ID
user_one_id	UUID	FK -> users.id, Indexed	First participant
user_two_id	UUID	FK -> users.id, Indexed	Second participant
created_at	TIMESTAMPTZ	Default NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	Default NOW(), Indexed	Timestamp of last message

Constraint: UNIQUE(user_one_id, user_two_id) where user_one_id < user_two_id to prevent duplicate conversation threads.

messages

Column	Type	Constraints	Description
id	BIGINT / UUID	Primary Key	Message identifier
conversation_id	UUID	FK -> conversations.id, Indexed	Parent conversation
sender_id	UUID	FK -> users.id, Indexed	Author of the message
content	TEXT	Not Null, Max 4000 chars	Encrypted / Raw text
status	ENUM	sent, delivered, read	Current lifecycle state

Column	Type	Constraints	Description
created_at	TIMESTAMPTZ	Default NOW(), Indexed	Message timestamp

7. Non-Functional Requirements (NFRs)

7.1 Performance & Latency

- **WebSocket Round-Trip Time:** Under 100ms for active connections on broadband/4G.
- **REST Response Time:** P95 response times < 150ms for auth and message history queries.
- **Concurrent Connections:** WebSocket Gateway capable of scaling to 100,000 concurrent socket connections via horizontal node scaling.

7.2 Security & Anti-Abuse

- **Brute-Force Protection:** Rate limit login attempts to 5 failed requests per IP/Username per 15 minutes. Implement exponential backoff lockouts.
- **Registration Throttling:** Restrict signups to a maximum of 3 accounts per IP address per hour to mitigate spam bot farms.
- **Transport Security:** All traffic strictly over HTTPS and TLS 1.3 (WSS).
- **Data Minimization:** No IP logging in application database tables. Ephemeral server logs purged every 7 days.

7.3 High Availability & Resilience

- 99.9% uptime target for the messaging gateway.
- Offline queuing: Messages sent to an offline recipient are persisted in the database and dispatched immediately upon socket reconnection.

8. UX / UI Key Wireframe Flows

```
[Flow 1: Fast Registration]
[ Landing Page ]
  |
  ▼ Click "Start Chatting"
[ Choose Username & Password ]
  |
  ▼ System generates 12-word Recovery Key
[ "Save Your Recovery Key" Modal (Mandatory Acknowledgment) ]
  |
  ▼
[ Main Chat Interface ]

[Flow 2: Starting a Chat]
[ Search Bar: Enter "@handle" ]
  |
```

```
└─ (Found) → Click "Message" → Open Blank Thread → Send
First Message
└─ (Not Found) → Display "No user found with username
'@handle'"
```

9. Phased Delivery Roadmap

Phase 1: MVP (Core Engine)

- Username + password registration with recovery key generation.
- Session authentication (JWT/Cookies) with rate-limiting.
- 1-on-1 text messaging via WebSockets with offline message persistence.
- Message status indicators (Sent, Delivered, Read) and Online/Offline presence.

Phase 2: Usability & Media

- Group chats (channels up to 100 users).
- End-to-end encryption (E2EE) using the Double Ratchet Algorithm (Signal Protocol).
- Ephemeral message self-destruct timers (e.g., delete after 1 hour / 24 hours).
- Basic image/file sharing (encrypted client-side before S3 upload).

Phase 3: Advanced Ecosystem

- Native desktop wrappers (Electron / Tauri) and mobile web PWA with push notifications.
- Audio/Video peer-to-peer calling via WebRTC.

10. Key Performance Indicators (KPIs)

1. **Onboarding Conversion Rate:** Percentage of users completing signup within 60 seconds of landing ($> 85\%$).
2. **Message Delivery Latency:** Median time from sender dispatch to recipient delivery ($< 100\text{ms}$).
3. **Session Retention:** Day 1 and Day 7 active conversation retention.
4. **Error Rate:** WebSocket dropped connection / dropped message packet rate ($< 0.05\%$).